

Sarcasm Detection in News Headlines Using RNNs and Word2Vec Embeddings

Student Name: Pratik Dhimal

Student ID: 2407779

Tutor: Ms. Sunita Parajuli

Submission Date: May 10, 2026

Part III - Language Task

GitHub Link:

<https://sl1nk.com/uiafjyw>

Table of Contents

- I Introduction 1**
- II Related Work 1**
- III Dataset 1**
- IV Methodology 1**
 - IV-A Text Preprocessing 1
 - IV-B Tokenization and Sequence Padding 2
 - IV-C Model Architectures 2
 - IV-C1 Simple RNN 2
 - IV-C2 LSTM 2
 - IV-C3 LSTM with Pre-trained Word2Vec 2
- V Results 2**
 - V-A Learning Curves 2
 - V-B Performance Summary 2
 - V-C Error Analysis 2
- VI Discussion 3**
- VII Conclusion 3**

Sarcasm Detection in News Headlines Using Recurrent Neural Networks: A Comparative Study of SimpleRNN, LSTM, and Word2Vec-Enhanced LSTM

Pratik Dhimal (ID: 2407779)

Abstract—Detecting sarcasm is a notoriously difficult task in NLP because it depends so much on tone and hidden context. In this project, I took a look at three different deep learning architectures—SimpleRNN, LSTM, and an LSTM using pre-trained Word2Vec vectors—to see how they handle headlines from the Onion and HuffPost. After cleaning the data and standardizing the text, I trained the models on over 28,000 headlines. I found that the standard LSTM performed the best, reaching an accuracy of 81.25%. Surprisingly, the Word2Vec model didn't beat the baseline, which suggests that sarcastic headlines have a specific vocabulary that general-purpose embeddings don't always capture.

Index Terms—sarcasm detection, natural language processing, LSTM, RNN, Word2Vec, text classification, news headlines

I. INTRODUCTION

Sarcasm is a type of verbal irony where what someone says is the opposite of what they actually mean. It is easy for people to spot, but it is a huge challenge for AI models used in sentiment analysis or fake news detection. If a model takes a sarcastic headline literally, it gets the entire sentiment wrong.

News headlines are a great place to study this because satirical outlets like *The Onion* write headlines that look like real news but use subtle jokes to signal sarcasm. For this project, I used a dataset that combines these sarcastic headlines with genuine ones from *HuffPost*. I wanted to build a pipeline that could accurately tell them apart using recurrent neural networks.

My main goals were to build a cleaning pipeline that handles the "noise" in short headlines, compare SimpleRNN, LSTM, and Word2Vec-LSTM under the same conditions, and figure out where these models fail through error analysis.

II. RELATED WORK

Originally, researchers used basic classifiers like SVMs with hand-picked features to find sarcasm. However, as more social media data became available, the focus shifted to deep learning. Ghosh and others showed that LSTMs are great at picking up the order of words to find ironic patterns. While newer models like BERT are very popular now, I wanted to focus on the fundamental RNN family to establish a solid baseline for how much sequence order matters for this specific dataset.

III. DATASET

The dataset I used has 28,619 labeled headlines. About 13,635 are sarcastic and 14,984 are genuine. Since the classes are pretty balanced (as shown in Fig. 1), I was able to use accuracy as my main metric without worrying about one class dominating the other.

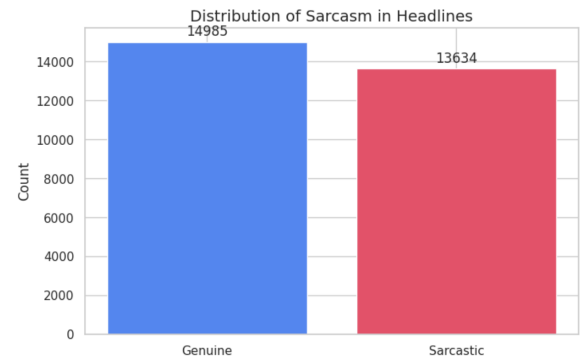


Fig. 1. Breakdown of sarcastic vs. genuine headlines.

IV. METHODOLOGY

A. Text Preprocessing

I found that news headlines are quite messy, so I followed these steps to clean them:

- 1) **Expanding Contractions:** I turned "don't" into "do not" so the model sees the full words.
- 2) **Cleaning:** I converted everything to lowercase and used regex to strip out URLs, mentions, hashtags, and numbers.
- 3) **Lemmaatization:** I reduced words to their base form (like "running" to "run").
- 4) **Stop-words:** I removed common English words that don't help with sentiment.

After cleaning, I generated word clouds (Fig. 2) to see the trends. Sarcastic headlines often used words like "nation" or "area" in hyperbolic ways, while genuine ones were more focused on real political figures and health topics.

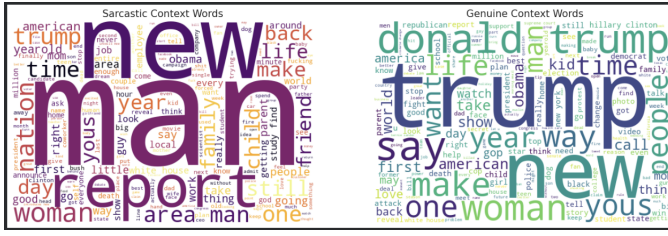


Fig. 2. Word clouds for sarcastic (left) and genuine (right) corpora.

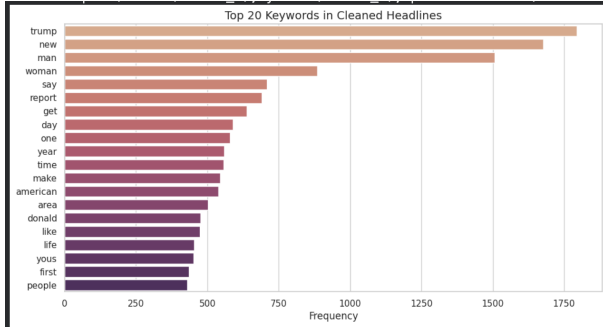


Fig. 3. The 20 most frequent words after I cleaned the data.

B. Tokenization and Sequence Padding

I used a Keras Tokenizer with a limit of 12,000 words. Any words the model hadn't seen before were marked as <UNK>. For padding, I looked at the distribution of token counts (Fig. 4) and chose the 95th percentile.

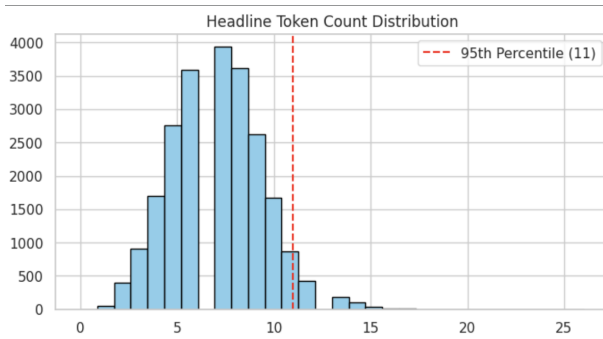


Fig. 4. Distribution of headline lengths; I padded sequences to the 95th percentile (red line).

C. Model Architectures

I kept the training settings consistent for all three models: Dropout at 0.3, a sigmoid output, and the Adam optimizer.

1) *Simple RNN*: This was my baseline. It had a single SimpleRNN layer with 64 units and trained its own embedding layer from scratch.

2) *LSTM*: I replaced the RNN with an LSTM layer. I expected this to do better because LSTMs have "gates" that help them remember context from the start of a sentence better than basic RNNs.

3) *LSTM with Pre-trained Word2Vec*: For this version, I used Google News vectors. I "froze" these embeddings during training so the model could rely on the pre-learned relationships between words.

V. RESULTS

A. Learning Curves

As I monitored the training (Figs. 5–7), the LSTM was clearly the most stable. The basic RNN was a bit jumpy, and while the Word2Vec model started strong, it didn't improve much after the first few epochs.

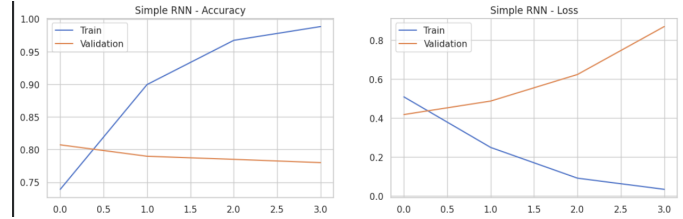


Fig. 5. Accuracy and loss for the Simple RNN.

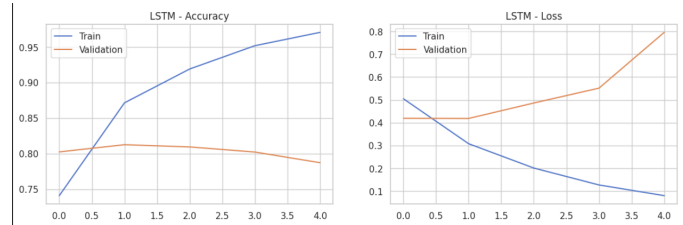


Fig. 6. Accuracy and loss for the LSTM.

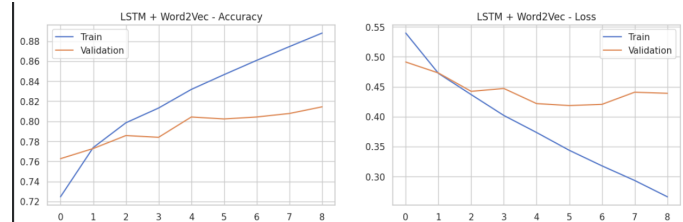


Fig. 7. Accuracy and loss for the Word2Vec model.

B. Performance Summary

Looking at the confusion matrices (Fig. 8), all models had a similar issue: they were slightly more likely to miss sarcasm (false negatives) than to flag a real headline as sarcastic (false positives).

The LSTM ended up being the winner across the board, as shown in Table I.

C. Error Analysis

I looked into the headlines the LSTM missed. There were two main patterns. First, the model struggled with "dry" sarcasm that didn't use obvious hyperbolic words. Second, it often flagged factual but "weird" headlines (like specific

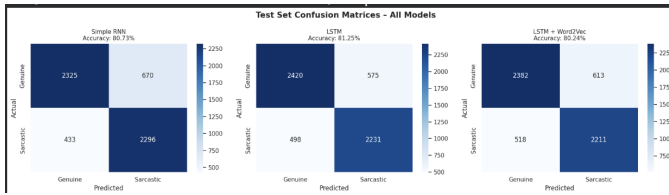


Fig. 8. Confusion matrices for all three models.

TABLE I
FINAL TEST-SET RESULTS

Model	Accuracy	Precision	Recall	F1
Simple RNN	80.73%	81.02%	80.73%	80.73%
LSTM	81.25%	81.30%	81.25%	81.26%
LSTM + Word2Vec	80.24%	80.30%	80.24%	80.25%

science news) as sarcastic because the vocabulary seemed surreal to the model.

VI. DISCUSSION

All three models performed similarly, hovering around 80-81%. This shows that sequence models are a great fit for this task. The Word2Vec model's performance suggests that domain-specific training data can sometimes outperform general-purpose embeddings if the task is niche.

VII. CONCLUSION

In this study, I compared SimpleRNN, LSTM, and Word2Vec-LSTM for sarcasm detection. The standard LSTM worked best with 81.25% accuracy. I learned that training your own embeddings from scratch is often better for sarcasm detection where the vocabulary is unique.



Fig. 9. Comparing metrics across the three models.